



DEVOPS CA-2

GARGI MITTAL 22070122064
GAUTAM RAJHANS 22070122068
ARYAN MALKANI 23070122502
YOGITA BENIWAL 23070122506

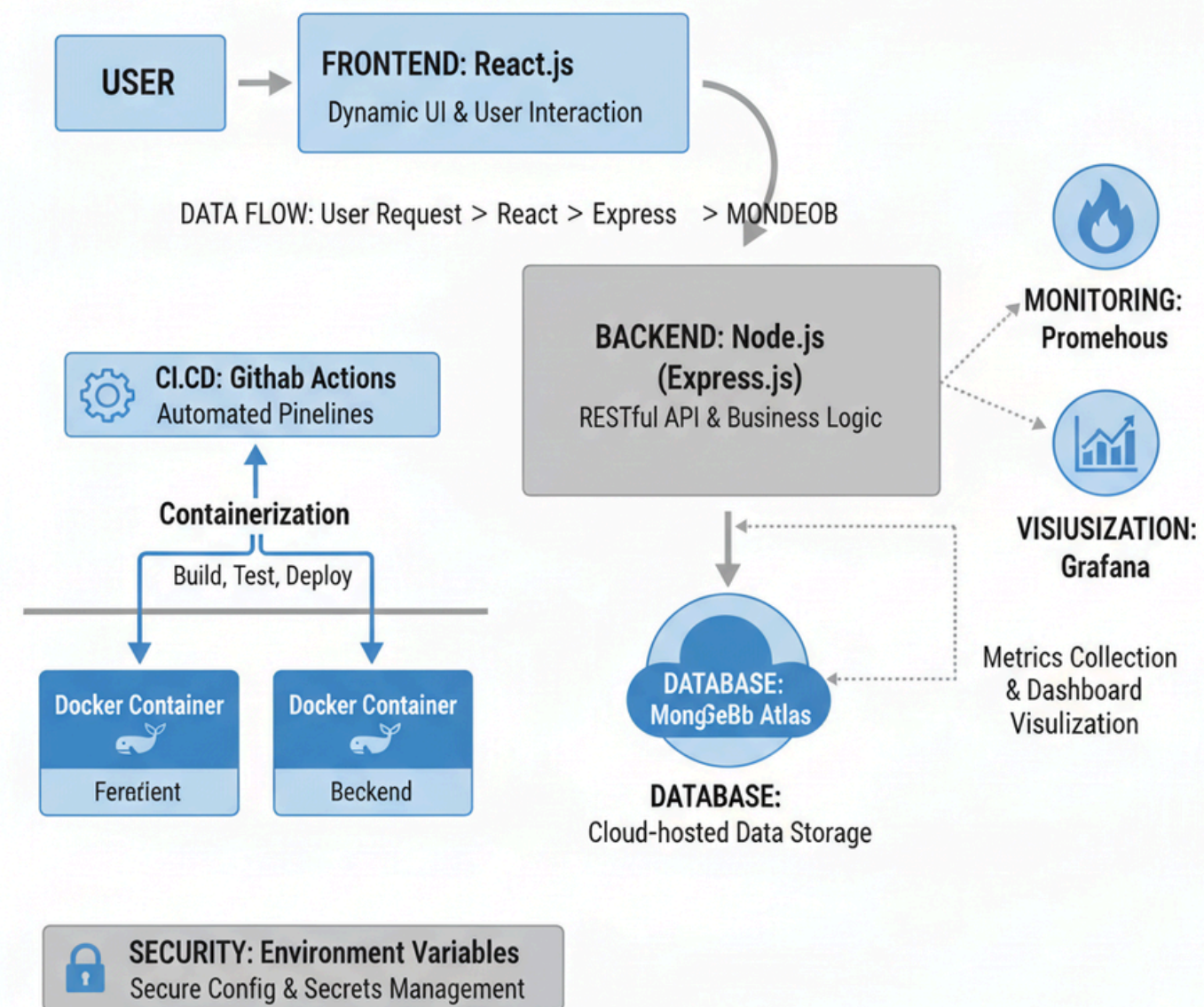
TO: ADITI SHARMA

TABLE OF CONTENTS

01	ARCHITECTURE
02	CI/CD PIPELINE
03	CHALLENGES
04	LESSONS LEARNED

ARCHITECTURE

FULL-STACK WEB APPLICATION SYSTEM ARCHIECTURE



Frontend: Built using React.js for dynamic UI rendering.

Backend: Node.js & Express.js for RESTful API and logic handling.

Database: MongoDB Atlas for cloud-based data storage.

DevOps: Docker containers ensure consistency across environments.

CI/CD: GitHub Actions automate build, test, and deployment.

Monitoring: Prometheus collects metrics, Grafana visualizes performance.

Secure configuration & secrets handled through environment variables.

PIPELINE FLOW

Code pushed to GitHub triggers automated workflow.

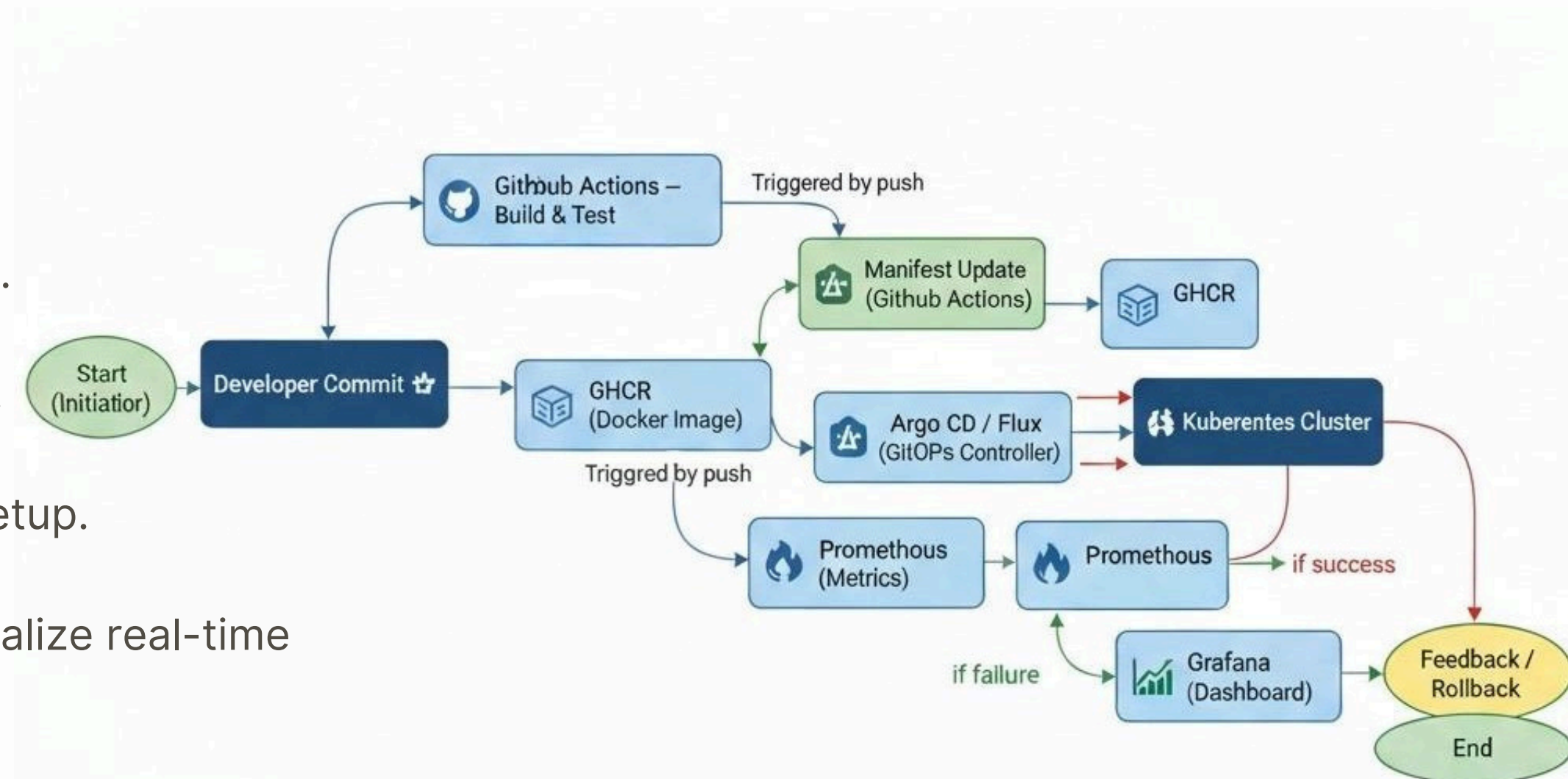
Build and test phase runs with dependency installation.

Docker image is built and pushed to container registry.

Deployed locally via Docker Compose for consistent setup.

Prometheus collects metrics, Grafana dashboards visualize real-time performance.

Feedback loop ensures continuous improvement and reliability.



CHALLENGES

Integration

Complexity in integrating Prometheus and Grafana dashboards.

Consistency

Maintaining Docker configurations across frontend and backend.

Security

Managing environment variables securely in CI/CD pipeline.

Monitoring

Setting up efficient monitoring with correct Prometheus targets.

Networking

Debugging container networking between services.

Compatibility

Handling version mismatches during image builds.

LESSONS LEARNED

1. Setting up efficient monitoring with correct Prometheus targets.

2. Docker ensures environment consistency across stages.

3. Monitoring helps in proactive debugging and optimization

4. Clear version control strategy prevents merge issues.

5. Documenting every stage simplifies team collaboration

6. Real-time observability improves reliability and performance.

THANK YOU